GO Live-Coding Cheatsheet

HackerRank-style interviews · syntax, std patterns, I/O, goroutines & channels, JSON · one A3 sheet

syntax key `keyword` `i64` `Type` `"str"` `macro!` `// comment`
★ use constantly · **TRAP** common gotcha · ▌ trap / tip

I/O 01–03 · Syntax 04–06 · Collections 07–11 · Types & Errors 12–14 · Concurrency 15–17 · JSON 18 · Algorithms 19–20

sections numbered **01→** in reading order, column by column

01 Fast I/O — bufio — never raw fmt.Scan in loops

I/O

```
package main
import ("bufio"; "fmt"; "os"; "strconv"; "strings")

func main() {
    in := bufio.NewReader(os.Stdin)          // buffered stdin ★
    out := bufio.NewWriter(os.Stdout)        // buffered stdout ★
    defer out.Flush()                        // or nothing is printed!
    var n int
    fmt.Fscan(in, &n)                        // whitespace-separated tokens
    a := make([]int, n)
    for i := range a { fmt.Fscan(in, &a[i]) }
    fmt.Fprintln(out, n, a[0])
}
// faster token reader for 10^6+ ints: bufio.Scanner + ScanWords
sc := bufio.NewScanner(os.Stdin)
sc.Buffer(make([]byte, 1<<20), 1<<20)        // raise 64KB line limit ★
sc.Split(bufio.ScanWords)
next := func() int { sc.Scan(); v, _ := strconv.Atoi(sc.Text()); return v }
n := next()
```

02 HackerRank Input Patterns — line-based stubs

I/O

```
sc := bufio.NewScanner(os.Stdin)             // line by line (default)
sc.Scan(); n, _ := strconv.Atoi(strings.TrimSpace(sc.Text()))
sc.Scan(); f := strings.Fields(sc.Text())    // "3 5" -> ["3", "5"]
a, _ := strconv.Atoi(f[0]); b, _ := strconv.Atoi(f[1])
// helper: one line -> []int
readInts := func() []int {
    sc.Scan(); f := strings.Fields(sc.Text()); v := make([]int, len(f))
    for i, s := range f { v[i], _ = strconv.Atoi(s) }
    return v
}
hdr := readInts(); n, m := hdr[0], hdr[1]
grid := make([][]int, n)                     // n rows of m ints
for i := range grid { grid[i] = readInts() }
g := make([][]byte, n)                       // "#..#" char grid -> bytes
for i := range g { sc.Scan(); g[i] = []byte(strings.TrimSpace(sc.Text())) }
// q command queries: "push 5" / "pop" / "print"
for i := 0; i < q; i++ {
    sc.Scan(); t := strings.Fields(sc.Text())
    switch t[0] {
    case "push": x, _ := strconv.Atoi(t[1]); st = append(st, x)
    case "pop":  st = st[:len(st)-1]
    default:     fmt.Fprintln(out, st[len(st)-1])
    }
}
for sc.Scan() { line := sc.Text(); if line == "" { continue } }  // until EOF
// output: space-joined ints, fixed decimals, YES/NO
strs := make([]string, len(v)); for i, x := range v { strs[i] = strconv.Itoa(x) }
fmt.Fprintln(out, strings.Join(strs, " "))
fmt.Fprintf(out, "%.6f\n", ans)
if ok { fmt.Fprintln(out, "YES") } else { fmt.Fprintln(out, "NO") }
```

Traps `fmt.Scan` is ~10× slower than `bufio.Scanner` default max line is 64KB → `sc.Buffer`. `Atoi` on `"5\r"` fails → `TrimSpace`. Sums beyond 2^{31}: `int` is 64-bit on amd64/arm64, so plain `int` is fine.

03 fmt · Printing

I/O

```
fmt.Println(a, b)              fmt.Printf("%d %s\n", n, s)
fmt.Printf("%v %+v %#v %T", x, p, p, x)   // value, +field names, Go syntax, type
fmt.Printf("%5d|%-5d|%05d|%x|%b|%o|%c|%q", n, n, n, n, n, n, c, s)
fmt.Printf("%.3f %8.2f %e %t %p", f, f, f, ok, &x)   // %*d: width from arg
fmt.Printf("%v %v %v", []int{1}, map[string]int{"a": 1}, struct{}{})
s := fmt.Sprintf("%d-%s", n, t)   fmt.Sprint(a, b)   fmt.Fprintln(os.Stderr, "e")
var sb strings.Builder; sb.WriteString("x"); sb.WriteByte('y'); sb.String() // ★
```

04 Functions · Closures · defer

SYNTAX

```
func add(a, b int) int { return a + b }
func divmod(a, b int) (q, r int) { q, r = a/b, a%b; return }  // named returns
func sum(nums ...int) int { }    sum(1, 2)   sum(v...)        // variadic
func apply(f func(int) int, x int) int { return f(x) }
sq := func(x int) int { return x * x }
cnt := 0;  inc := func() { cnt++ }     // closures capture by reference ★
var dfs func(u int)                    // recursive closure: declare first ★
dfs = func(u int) { vis[u] = true
    for _, v := range adj[u] { if !vis[v] { dfs(v) } } }
defer f.Close()                        // runs at function return, LIFO order
defer func() { if r := recover(); r != nil { log.Println(r) } }()
func Map[T, U any](xs []T, f func(T) U) []U {          // generics ★
    out := make([]U, 0, len(xs)); for _, x := range xs { out = append(out, f(x)) }
    return out }
func Max[T cmp.Ordered](a, b T) T { if a > b { return a }; return b }
type Number interface { ~int | ~int64 | ~float64 }   // constraint
```

Trap Loop var in closures: since Go 1.22 each iteration has its own `i`; before, `go func(){ use(i) }()` saw the last value — pass `i` as an arg.

05 Control Flow

SYNTAX

```
if x > 0 { } else if x < 0 { } else { }        // braces required, no parens
if v, ok := m[k]; ok { }                       // init statement ★
for i := 0; i < n; i++ { }                     // the only loop keyword
for i := 1; i <= 10; i++ { }                   // 1..10 inclusive ★
for i := n - 1; i >= 0; i-- { }
for i := range n { }                           // 0..n-1  (Go 1.22+) ★
for i, v := range a { }   for _, v := range a { }   for i := range a { }
for k, v := range m { }   for i, r := range s { }   // s: rune + byte index
for cond { }  /* while */    for { if done { break } }  /* loop */
outer: for i := range n { for j := range m { if bad { continue outer } } }
switch x {
case 0:        s = "zero"
case 1, 2:     s = "small"                     // no fallthrough by default
case 3, 4, 5:  fallthrough                     // explicit
default:       s = "big"
}
switch { case x < 0: ..; case x < 10: ..; default: .. }   // if-else chain ★
switch v := x.(type) { case int: ..; case string: ..; case nil: .. } // type switch
```

06 Variables · Types · Casting

SYNTAX

```
var x int = 5;  var y = 10;  z := 3.0        // := only inside funcs (new var)
var a, b int;  c, d := 1, "hi";  a, b = b, a  // swap ★
const Mod = 1_000_000_007;  const ( A = iota; B; C )   // 0, 1, 2
x := int64(y) * int64(z)          // conversion is explicit, no implicit widening ★
f := float64(n) / float64(m)      // int / int truncates
i := int(f)                       // truncates toward 0;  math.Round(f)
r := rune(s[0]);  b := byte('a') // string index = byte;  "é" is 2 bytes
d := int(c - '0')                 // '7' -> 7;   string(rune('a' + i))
n, err := strconv.Atoi(s);  s := strconv.Itoa(n)          // string <-> int ★
f, _ := strconv.ParseFloat(s, 64);  strconv.FormatInt(n, 2)  // base 2
math.MaxInt64  math.MinInt64  math.MaxInt  math.Inf(1)  math.MaxFloat64
x % m                             // sign follows x: (-7) % 3 == -1 → ((x%m)+m)%m
math.Abs(f) math.Sqrt(f) math.Pow(f, 2) math.Floor/Ceil/Round math.Log2(f)
min(a, b)  max(a, b, c)           // builtins (Go 1.21+) ★   abs: write it
```

Trap Unused variable or import = compile error. `int` overflow wraps silently (no panic). x++ is a statement, not an expression.

07 Slices

COLLECTIONS

```
var s []int;  s := []int{1, 2, 3};  s := make([]int, n)  /* n zeros */
s := make([]int, 0, n)            // len 0, cap n (append without realloc) ★
g := make([][]int, n); for i := range g { g[i] = make([]int, m) }  // 2D grid ★
s = append(s, x);  s = append(s, a, b);  s = append(s, t...)   // always reassign ★
len(s)  cap(s)  s[i]  s[len(s)-1]  s[1:3]  s[:k]  s[k:]  s[:]
s = s[:len(s)-1]                  // pop
s = append(s[:i], s[i+1:]...)     // remove i (shifts; shares backing array)
s = append(s[:i], append([]int{x}, s[i:]...)...)                  // insert at i
s[i], s[j] = s[j], s[i]           // swap
copy(dst, src)                    // copies min(len) elements, returns count
c := make([]int, len(s)); copy(c, s)    /* clone */    c := slices.Clone(s)
slices.Reverse(s)  slices.Contains(s, x)  slices.Index(s, x)  slices.Max(s)
slices.Sort(s)
slices.SortFunc(s, func(a, b T) int { return cmp.Compare(a.k, b.k) })
slices.BinarySearch(s, x)  // (idx, found)     slices.Equal(a, b)
slices.Insert(s, i, x)  slices.Delete(s, i, j)  slices.Compact(s)  /* dedup */
```

Trap `s[a:b]` shares memory: appending to a sub-slice can overwrite the parent. Copy when in doubt. `nil` slice is fine with `len` / `append` / `range`.

08 Array ↔ Slice · Conversion · Concat

COLLECTIONS

```
var a [26]int;  a := [3]int{1, 2, 3};  a := [...]int{1, 2}     // fixed, value type
cnt := [26]int{}; cnt[c-'a']++          // letter counts ★;  arrays copy on assign!
s := a[:]                               // array -> slice (shares memory)
var b [3]int = [3]int(s)                // slice -> array (Go 1.20+, len must match)
p := (*[3]int)(s)                       // slice -> array pointer (shares)
// concatenation: append(dst, src...) is the one idiom ★
c := append(a, b...)                    // MAY alias a if cap(a) suffices ★
c := append([]int{}, a...);  c = append(c, b...)      // safe: fresh backing
c := slices.Concat(a, b, d)             // Go 1.22+: always new slice ★
c := append(a[:len(a):len(a)], b...)    // full slice expr: force realloc
flat := slices.Concat(grid...)          // [][]T -> []T
strings.Join(words, " ")    strings.Split(s, ",")   strings.Fields(s)
[]byte(s)   string(bs)   []rune(s)   string(rs)    // string <-> bytes/runes
strings.Repeat("ab", 3)   bytes.Repeat([]byte("ab"), 3)
```

09 Maps · Sets

COLLECTIONS

```
m := map[string]int{};  m := make(map[string]int, n);  m := map[string]int{"a": 1}
m[k] = v;  v := m[k]  /* zero value if missing ★ */   delete(m, k)   len(m)
v, ok := m[k]                   // ok: present?      if _, ok := m[k]; !ok { }
m[k]++                          // counting ★ (missing -> 0 -> 1)
m[k] = append(m[k], x)   // adjacency list ★ (nil slice append is fine)
for k, v := range m { }  // random order! sort keys for determinism:
keys := make([]string, 0, len(m)); for k := range m { keys = append(keys, k) }
sort.Strings(keys)              // or slices.Sorted(maps.Keys(m))   (Go 1.23)
clear(m)                        // Go 1.21
// set = map[T]struct{} (0 bytes) or map[T]bool
seen := map[int]struct{}{};  seen[x] = struct{}{};  _, has := seen[x]
seen := map[int]bool{};  seen[x] = true;  if seen[x] { }              // simpler
// map keys must be comparable: struct{x, y int}, [2]int ok; slices NOT
dist := map[[2]int]int{};  dist[[2]int{r, c}] = 0
type P struct{ X, Y int };  m := map[P]bool{};  m[P{1, 2}] = true
```

Trap `var m map[K]V` is nil: reading is fine, writing PANICS → use `make`. Map values are not addressable: `m[k].field = 1` won't compile — copy, modify, store back (or store pointers).

10 Strings · Bytes · Runes

COLLECTIONS

```
s := "héllo";  len(s)  /* bytes: 6 */    utf8.RuneCountInString(s)  /* chars: 5 */
s[i]                            // byte (uint8), NOT char;  s[1:3] byte slice
for i, r := range s { }  // decodes runes; i = byte offset
rs := []rune(s);  rs[1]     // index by char ★;  string(rs)
bs := []byte(s);  bs[0] = 'H';  s = string(bs)      // strings are immutable
s + t   fmt.Sprintf   strings.Builder (loops)   strings.Join(parts, "")
strings.Contains(s, "x")  .HasPrefix  .HasSuffix  .Index(s, "x")  /* -1 */
strings.ToUpper  .ToLower  .TrimSpace  .Trim(s, "ab")  .TrimLeft  .TrimPrefix
strings.Split(s, ",")  .Fields(s)  .SplitN(s, ":", 2)
before, after, found := strings.Cut(s, "=")
strings.Replace(s, "a", "b", -1)  .ReplaceAll  .Repeat  .Count  .EqualFold
strings.Compare(a, b)   a < b   // lexicographic byte compare works directly
// reverse a string (by runes)
rs := []rune(s)
for i, j := 0, len(rs)-1; i < j; i, j = i+1, j-1 { rs[i], rs[j] = rs[j], rs[i] }
unicode.IsLetter(r)  .IsDigit  .IsSpace  .IsUpper  .ToLower(r)   // for runes
'a' <= c && c <= 'z'    c - '0'    byte('a' + i)      // bytes, ASCII fast path ★
strconv.Quote(s)   strconv.ParseBool("true")   strconv.AppendInt(buf, n, 10)
```

11 sort · container/heap · list

COLLECTIONS

```
sort.Ints(a)  sort.Strings(s)  sort.Float64s(f)        // ascending, in place
sort.Sort(sort.Reverse(sort.IntSlice(a)))              // descending
sort.Slice(ps, func(i, j int) bool { return ps[i].age < ps[j].age })   // ★
sort.SliceStable(ps, func(i, j int) bool {       // multi-key: score desc, name asc
    if ps[i].s != ps[j].s { return ps[i].s > ps[j].s }; return ps[i].n < ps[j].n })
slices.SortFunc(ps, func(a, b P) int {                 // Go 1.21 style, faster
    return cmp.Or(cmp.Compare(b.s, a.s), cmp.Compare(a.n, b.n)) })
i := sort.SearchInts(a, x)                             // lower_bound ★
i := sort.Search(n, func(i int) bool { return a[i] >= x })  // first i: pred true
// min-heap via container/heap: implement 5 methods ★
type IntHeap []int
func (h IntHeap) Len() int           { return len(h) }
func (h IntHeap) Less(i, j int) bool { return h[i] < h[j] }   // > for max-heap
func (h IntHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }
func (h *IntHeap) Push(x any)        { *h = append(*h, x.(int)) }
func (h *IntHeap) Pop() any { o := *h; n := len(o); x := o[n-1]
    *h = o[:n-1]; return x }
h := &IntHeap{3, 1}; heap.Init(h); heap.Push(h, 2)
x := heap.Pop(h).(int);  (*h)[0] /* peek */   h.Len()
// queue / deque: a slice is enough  q = append(q, x); x, q = q[0], q[1:]
```

12 Struct · Methods · Interfaces

TYPES & ERRORS

```
type P struct { X, Y int; Name string `json:"name"` }
p := P{1, 2, "a"};  p := P{X: 1};  pp := &P{X: 1};  var z P  /* zero value */
p.X  pp.X  /* auto-deref */    (*pp).X   p2 := p  /* copy! structs are values */
func (p P) Dist() int { return abs(p.X) + abs(p.Y) }     // value receiver: copy
func (p *P) Shift(d int) { p.X += d }                    // pointer receiver: mutates ★
p.Shift(1)   // Go auto-takes &p;  but a map value / slice elem must be addressable
type Stack struct{ items []int }                         // embedding = composition
type Named struct{ P; Tag string };  n.X   n.P.X         // promoted fields
type Shape interface { Area() float64 }                  // implicit implementation ★
type Circle struct{ R float64 }
func (c Circle) Area() float64 { return 3.14 * c.R * c.R }
var s Shape = Circle{2}                                  // no "implements" keyword
shapes := []Shape{Circle{1}, Rect{2, 3}}
if c, ok := s.(Circle); ok { }                           // type assertion
func (p P) String() string { return fmt.Sprintf("(%d,%d)", p.X, p.Y) }  // Stringer
var _ Shape = (*Circle)(nil)                             // compile-time check
type Color int;  const ( Red Color = iota; Green; Blue )  // enum
func (c Color) String() string { return [...]string{"R", "G", "B"}[c] }
```

Trap A method with a pointer receiver means only `*T` satisfies the interface, not `T`. Interface holding a typed nil pointer is NOT `== nil`.

13 Pointers · Values · Zero Values

TYPES & ERRORS

```
var p *int;  p = &x;  *p = 5;  p == nil          // no pointer arithmetic
q := new(int)          // *int pointing at 0;   &T{} idiom for structs ★
func inc(n *int) { *n++ }    inc(&x)             // pass pointer to mutate
func f(s []int) { s[0] = 9 }  // slices/maps/chans share data: mutation visible ★
func g(s []int) { s = append(s, 1) }  // but append may not be visible: return it
// zero values: 0  ""  false  nil (ptr/slice/map/chan/func/interface)  struct{..}
var wg sync.WaitGroup; var mu sync.Mutex; var sb strings.Builder // zero = ready ★
a == b   // structs/arrays comparable if all fields are; slices/maps only vs nil
reflect.DeepEqual(a, b)   slices.Equal(a, b)   maps.Equal(a, b)
```

14 Errors · panic · recover

TYPES & ERRORS

```
func parse(s string) (int, error) {
    n, err := strconv.Atoi(s)
    if err != nil { return 0, fmt.Errorf("parse %q: %w", s, err) }  // wrap: %w ★
    if n < 0 { return 0, errors.New("negative") }
    return n, nil
}
n, err := parse(s); if err != nil { return err }     // the idiom: check & return ★
var ErrNotFound = errors.New("not found")                  // sentinel
if errors.Is(err, ErrNotFound) { }                         // matches through wraps
var pe *os.PathError; if errors.As(err, &pe) { pe.Path }   // extract typed error
type MyErr struct{ Code int };  func (e *MyErr) Error() string { return "..." }
errors.Join(e1, e2)   err.Error()   log.Fatal(err)  /* exit 1 */   os.Exit(1)
panic("bad")   // unwinds, runs defers;  index out of range / nil map write panic
defer func() { if r := recover(); r != nil { err = fmt.Errorf("rec: %v", r) } }()
_ = x   // discard;   _, err := f()   // ignore value, keep error
```

15 Goroutines · Channels

CONCURRENCY

```
go f(x)   go func() { ... }()        // lightweight thread; main exits = all die ★
ch := make(chan int)                 // unbuffered: send blocks until received
ch := make(chan int, 16)             // buffered: send blocks only when full
ch <- v                              // send       v := <-ch   // receive (blocks)
v, ok := <-ch                        // ok=false when closed & drained
close(ch)                            // only the SENDER closes ★; send after: panic
for v := range ch { }                // loops until closed ★
var send chan<- int = ch;  var recv <-chan int = ch   // direction-typed params
select {                             // wait on multiple channels
case v := <-a:      use(v)
case b <- x:        sent()
case <-time.After(time.Second): timeout()
case <-ctx.Done():  return ctx.Err()
default:            nonBlocking()  // only if nothing is ready
}
var wg sync.WaitGroup
for i := range n { wg.Add(1); go func() { defer wg.Done(); work(i) }() }
wg.Wait()                            // Go 1.25: wg.Go(func(){ work() })
var mu sync.Mutex; mu.Lock(); cnt++; mu.Unlock()   // or defer mu.Unlock()
var n atomic.Int64; n.Add(1); n.Load()             // lock-free counter
var once sync.Once; once.Do(initFn)   sync.RWMutex  RLock/RUnlock
```

Traps Deadlock: "all goroutines are asleep" — unbuffered send with no receiver, or `range` on a channel nobody closes. Reading a nil channel blocks forever. Run with `-race`.

16 Producer → Multiple Consumers — worker pool ★

CONCURRENCY

```
func worker(id int, jobs <-chan int, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs {                        // each job goes to exactly one worker
        results <- Result{id, j, j * j}          // channels are MPMC-safe by design ★
    }
}
func main() {
    jobs := make(chan int, 100)
    results := make(chan Result, 100)
    var wg sync.WaitGroup
    for w := 1; w <= 4; w++ {                    // N consumers
        wg.Add(1)
        go worker(w, jobs, results, &wg)
    }
    go func() {                                  // 1 producer
        for i := 0; i < 20; i++ { jobs <- i }
        close(jobs)                              // signals workers: no more work ★
    }()
    go func() { wg.Wait(); close(results) }() // close results after workers exit ★
    for r := range results { fmt.Println(r) } // consume until closed
}
// variants
sem := make(chan struct{}, 4)                    // semaphore: limit concurrency to 4
for _, j := range jobs {
    sem <- struct{}{}; go func() { defer func() { <-sem }(); do(j) }() }
// bounded fan-in with errgroup (golang.org/x/sync):
g, ctx := errgroup.WithContext(ctx); g.SetLimit(4)
for _, j := range jobs { g.Go(func() error { return do(ctx, j) }) }
if err := g.Wait(); err != nil { }               // first error cancels ctx
// broadcast to ALL consumers (not a queue): close(done) wakes every <-done ★
```

Order ① start workers ② producer sends + closes jobs ③ wg.Wait then close results ④ range results. Closing `results` from a worker = panic when another worker sends.

17 Pipelines · Fan-out / Fan-in · Context

CONCURRENCY

```
// stage: in-channel -> out-channel, each stage a goroutine
func gen(nums ...int) <-chan int { out := make(chan int)
    go func() { defer close(out); for _, n := range nums { out <- n } }()
    return out }
func sq(in <-chan int) <-chan int { out := make(chan int)
    go func() { defer close(out); for n := range in { out <- n * n } }()
    return out }
for v := range sq(sq(gen(1, 2, 3))) { }                    // compose
// fan-out: run N copies of a stage on the same input; fan-in: merge their outputs
func merge(cs ...<-chan int) <-chan int {
    out := make(chan int); var wg sync.WaitGroup; wg.Add(len(cs))
    for _, c := range cs {
        go func() { defer wg.Done(); for v := range c { out <- v } }() }
    go func() { wg.Wait(); close(out) }()
    return out }
c1, c2 := sq(in), sq(in); for v := range merge(c1, c2) { }
// cancellation & timeouts with context ★
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()
ctx, cancel := context.WithCancel(ctx)        // cancel() to stop all listeners
select { case out <- v: case <-ctx.Done(): return ctx.Err() }  // cancellable send
// collect results from N goroutines in order (distinct slice index = safe)
res := make([]int, n); var wg sync.WaitGroup
for i := range n { wg.Add(1); go func() { defer wg.Done(); res[i] = f(i) }() }
wg.Wait()
time.Sleep(10 * time.Millisecond)  time.Now()  time.Since(t0)  time.NewTicker(d)
```

18 JSON · encoding/json — struct tags drive everything

JSON

```
type User struct {
    ID    int       `json:"id"`
    Name  string    `json:"name"`
    Email *string   `json:"email,omitempty"`   // nil -> omitted; null -> nil
    Tags  []string  `json:"tags,omitempty"`    // missing -> nil slice
    Kind  string    `json:"type"`              // rename
    pw    string                               // unexported: never (de)serialized
    Skip  string    `json:"-"`
}
var u User; err := json.Unmarshal([]byte(s), &u)   // bytes -> struct (pointer!) ★
var us []User; json.Unmarshal(data, &us)           // JSON array
b, _ := json.Marshal(u);  string(b)                // struct -> bytes
b, _ := json.MarshalIndent(u, "", "  ")            // pretty
json.NewDecoder(os.Stdin).Decode(&u)               // from reader (stdin/file/http)
json.NewEncoder(w).Encode(u)                       // to writer
// untyped: map / any
var v map[string]any; json.Unmarshal(data, &v)
v["a"].(map[string]any)["b"].([]any)[0].(float64)  // numbers are float64! ★
if arr, ok := v["tags"].([]any); ok { }
var raw map[string]json.RawMessage                 // defer parsing of fields
m := map[string]any{"id": 1, "ok": true, "n": nil}; json.Marshal(m) // keys sorted
// custom (un)marshal
func (c Color) MarshalJSON() ([]byte, error) { return json.Marshal(c.String()) }
func (c *Color) UnmarshalJSON(b []byte) error { var s string; .. }
dec := json.NewDecoder(r); for dec.More() { var m Msg; dec.Decode(&m) }  // NDJSON
```

Trap Fields must be exported (Capitalized) or they're silently ignored. `Unmarshal` into a non-pointer silently does nothing useful. Unknown fields are ignored unless `dec.DisallowUnknownFields()`.

19 Algorithm Templates

ALGORITHMS

```
// BFS on a grid
type pt struct{ r, c int }
dirs := [4]pt{{0, 1}, {1, 0}, {0, -1}, {-1, 0}}
dist := make([][]int, n)
for i := range dist { dist[i] = slices.Repeat([]int{-1}, m) }
dist[sr][sc] = 0; q := []pt{{sr, sc}}
for len(q) > 0 {
    cur := q[0]; q = q[1:]
    for _, d := range dirs {
        nr, nc := cur.r+d.r, cur.c+d.c
        if nr < 0 || nc < 0 || nr >= n || nc >= m { continue }
        if grid[nr][nc] == '#' || dist[nr][nc] != -1 { continue }
        dist[nr][nc] = dist[cur.r][cur.c] + 1; q = append(q, pt{nr, nc})
    }
}
// Dijkstra with container/heap (type Item struct{d, u int}; Less by d)
dist := slices.Repeat([]int{math.MaxInt}, n); dist[s] = 0
pq := &PQ{{0, s}}; heap.Init(pq)
for pq.Len() > 0 {
    it := heap.Pop(pq).(Item); if it.d > dist[it.u] { continue }
    for _, e := range adj[it.u] { if nd := it.d + e.w; nd < dist[e.v] {
        dist[e.v] = nd; heap.Push(pq, Item{nd, e.v}) } }
}
// Union-Find
par := make([]int, n); for i := range par { par[i] = i }
var find func(x int) int
find = func(x int) int { if par[x] != x { par[x] = find(par[x]) }; return par[x] }
union := func(a, b int) { a, b = find(a), find(b); if a != b { par[a] = b } }
// binary search on answer: smallest x in [lo, hi) with ok(x)
lo, hi := 0, int(1e18)
for lo < hi { mid := lo + (hi-lo)/2; if ok(mid) { hi = mid } else { lo = mid+1 } }
// DP table · prefix sum · sliding window
dp := make([][]int, n+1); for i := range dp { dp[i] = make([]int, m+1) }
pre := make([]int, n+1); for i, x := range a { pre[i+1] = pre[i] + x } // prefix
l, sum := 0, 0; for r, x := range a { sum += x; for sum > k { sum -= a[l]; l++ } }
memo := map[int]int{}   // memoized recursion: map + recursive closure (see dfs)
func gcd(a, b int) int { for b != 0 { a, b = b, a%b }; return a }
func pw(b, e, m int) int { r := 1; b %= m
    for e > 0 { if e&1 == 1 { r = r * b % m }; b = b * b % m; e >>= 1 }; return r }
```

20 Misc · Easy to Forget

ALGORITHMS

```
bits.OnesCount(uint(x)) bits.Len(uint(x)) bits.TrailingZeros(x)   math/bits; x & (x - 1)
1 << k  x >> 1  x &^ y                                  shift · and-not (bit clear)
os.Args[1:] os.ReadFile(p) os.WriteFile(p, b, 0644)     CLI args · read / write a file
x, y = y, x  a[i], a[j] = a[j], a[i]                    tuple assignment swap
if _, ok := m[k]; ok { } // scoped ok                   shadowing-safe check
[]int(nil) len == 0 append works                        nil slice is a valid empty slice
go run . && go build -o app . && go test ./...          run · build · test
go vet ./... go run -race . gofmt -w .                  vet · race detector · format
```

Routine ① `bufio.NewReader` + `NewWriter` + `defer Flush()` ② parse all input first ③ `int` is 64-bit — no overflow worries below 9e18 ④ `append` → reassign ⑤ unused var? `_ = x` ⑥ `go run -race` when goroutines misbehave.